arXiv:2611.10463v1 [cs.CL] 7 Nov 2026

LongCapHarness: Capability-Guided Auto-Harnessing for Long-Context Reasoning

Tao Zhang
Microsoft Research Asia
zhangt@microsoft.com

Yuhang He*
Microsoft Research Asia
yuhanghe@microsoft.com

Abstract

Existing methods enhancing an LLM's long-context reasoning capability often rely on data synthesis and follow-up LLM post-training. However, the data synthesis strategy and the post-training recipes are empirically driven and computationally intensive. In this work, we propose a new framework, dubbed *LongCapHarness*, to instead harness the underlying capabilities for long-context reasoning. The harnessing process recursively revises and updates capability primitive implementation, capability state and control logic through recursive self-improvement on the development set. With neither data synthesis nor LLM post-training, *LongCapHarness* is applied to Llama3-8B and transferred to Qwen2.5-3B. *LongCapHarness* substantially outperforms recent data-synthesis-based baselines on HELMET and RULER benchmarks; its estimated development cost is less than 50% of the estimated training costs of NExtLong and PolicyLong. We hope these results motivate further research into harnessing the latent capabilities of LLMs to solve tasks beyond their explicit training.

1 Introduction

Existing methods for enhancing a pretrained LLMs' long-context reasoning capabilities follow the data synthesis pipeline: synthesizing a large long-context corpus, followed by post-training the LLM on the synthesized data (*e.g.*, NExtLong (Gao et al., 2025b), PolicyLong (Jia et al., 2026b), LongFilter (Deng et al., 2026), EntropyLong (Jia et al., 2025), LADM (Chen et al., 2025), Quest (Gao et al., 2025a), LiteLong (Jia et al., 2026a)). Despite the observed gains on long-context benchmarks such as HELMET (Yen et al., 2025) and LongBench-v2 (Bai et al., 2025), these methods disentangle data synthesis from the follow-up LLM post-training, and the data synthesis strategies and post-training recipes are empirically motivated and computationally intensive. Therefore, the direction on data synthesis and LLM post-training is costly and difficult to scale-up.

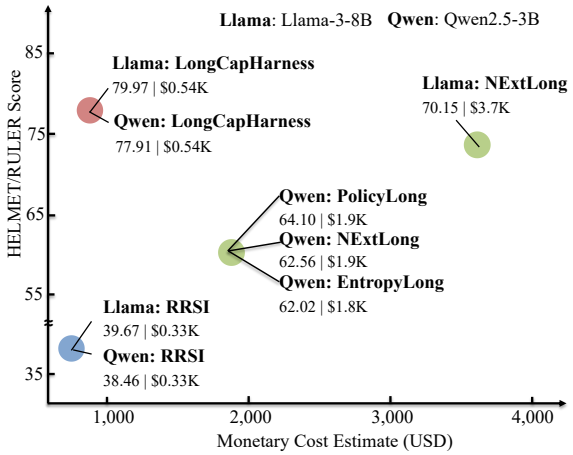

Figure 1: Performance superiority and cost effectiveness of *LongCapHarness* compared with the three recent data-synthesis-based baselines and the RSI baseline.

In this work, we propose an alternating framework, dubbed *LongCapHarness*, that substantially outperforms data-synthesis-based methods while incurring much lower development cost (see Fig. 1). *LongCapHarness* builds on the key observation:

> *LLMs without prior explicit long-context pre-training can still show strong performance on long-context benchmarks as long as they have been appropriately equipped with long-context reasoning capability.*

To build up such a framework, we leverage the latest harness engineering technologies (Chen et al., 2026; Lee et al., 2026; Lin et al., 2026; Lou et al., 2026; Nie et al., 2026) to design a long-context capability-guided auto-harnessing framework. We argue that the central challenge in long-context reasoning is not merely fitting a long input within the

*Corresponding author.

available context window. Solving such a task often requires identifying its task structure, localizing relevant evidence, selectively retaining intermediate results, and finally integrating them into a globally consistent answer. In a monolithic end-to-end call, these operations and dependencies remain implicit, making it hard for the model to coordinate them while processing the full input at once. To address this challenge, we propose a capability harness to support long-context tasks. Capability decomposition further makes this latent task structure explicit by breaking the long-context task into operations with defined inputs and outputs. The harness can then preserve shared state across operations and supply each model call only with the context it needs, decoupling total input length from the context budget of any individual call.

In *LongCapHarness*, we leverage a harnessing agent to construct reusable capability harnesses from the development set. The harness agent studies the task input to identify the required capability, then decomposes it into capability primitives. The harnessing process recursively revises and updates the primitive implementation, capability state and control logic by comparing the answer generated by running the capability harness against the reference answer. The final capability harness is frozen and saved to a library. At test time, an execution LLM selects and runs the appropriate capability harness on the held-out test set.

We evaluate *LongCapHarness* on HELMET (Yen et al., 2025), RULER (Hsieh et al., 2024), and LongBench-v2 (Bai et al., 2025), using GPT-5.6-Sol (OpenAI, 2026) for harness development and Llama3-8B (Meta, 2024) for execution before transferring the harness to Qwen2.5-3B (Qwen et al., 2024). On HELMET and RULER, it outperforms the corresponding model-training baselines (Gao et al., 2025b; Jia et al., 2026b) and RRSI (Xia et al., 2026) with both execution models. On LongBench-v2, Qwen2.5-3B with the harness surpasses PolicyLong (Jia et al., 2026b) on the Medium subset but trails it on the Long subset.

Estimated development API costs are lower than the estimated training costs of NExtLong and PolicyLong, but higher than RRSI development costs. At inference, *LongCapHarness* uses substantially fewer output tokens than RRSI; estimated costs are comparable with Llama but higher with Qwen, where increased input-token use offsets the output savings.

Our contributions are summarized as follows:

1. We propose *LongCapHarness*, a novel framework for long-context reasoning tasks with neither long-context data synthesis nor LLM post-training.
2. We demonstrate the superiority of *LongCapHarness* in terms of performance on benchmarks and cost effectiveness via extensive experiments.
3. We hope to motivate more research into harnessing the latent capabilities of LLMs to solve complex tasks that go beyond their explicit training.

2 Related Work

Data-Synthesis-based Long-Context Reasoning. Most recent methods on improving long-context reasoning first synthesize long-context data and then use it to post-train LLMs (Jia et al., 2026b, 2025; Gao et al., 2025b; Chen et al., 2025; Gao et al., 2025a). These data strategies are often developed empirically and separately from the post-training recipes. This separation makes it difficult to determine which data properties are essential for long-context reasoning and how data preparation should be aligned with post-training. For example, ProLong (Gao et al., 2025c) studies training recipes and data mixtures, while NExtLong (Gao et al., 2025b) constructs long sequences with hard-negative distractors. EntropyLong (Jia et al., 2025) verifies useful context through predictive uncertainty, and PolicyLong (Jia et al., 2026b) refreshes data using the evolving model. *LongCapHarness* instead proposes an alternating direction with neither data synthesis nor LLM post-training. Benefiting from its carefully designed capability harnessing framewor, *LongCapHarness* exhibits both performance superiority and cost effectiveness compared with these data-synthesis-based methods.

Model–program computation and semantic operators. Model–program systems combine structured computation with learned semantic operations. SUQL (Liu et al., 2024) extends SQL with free-text question answering and summarization, while LOTUS (Patel et al., 2025) provides optimized semantic operators for filtering, joining, aggregation, and ranking. *LongCapHarness* uses task capabilities to guide how computational inputs are recovered from long documents and composed into executable workflows.

Agent and harness design. Automated agent design improves model-based systems by searching and refining external execution logic. ADAS (Hu et al., 2024) searches over code-defined agents using past designs and evaluations. AFlow (Zhang et al., 2024) optimizes workflows through Monte Carlo Tree Search and execution feedback, and AutoHarness (Lou et al., 2026) synthesizes harnesses and policies from game-environment feedback. *LongCapHarness* adds explicit capability contracts to guide the construction and refinement of representations, operators, and model calls for long-context reasoning.

3 *LongCapHarness* Introduction

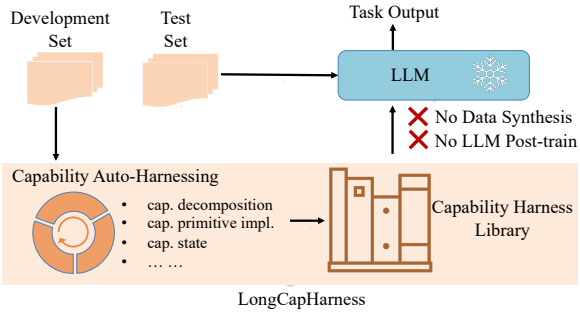

Figure 2: *LongCapHarness* uses recursive self-improvement (RSI) on the development set to construct and refine reusable capability harnesses. The resulting library is used at test time to assist an LLM execution on long-context tasks. *LongCapHarness* requires neither data synthesis nor LLM post-training.

LongCapHarness starts with a development set of task inputs and corresponding reference answers. The harnessing agent (in our case, we adopt GPT-5.6-Sol (OpenAI)) studies the task, identifies the required capability, and further decomposes it into capability primitives (Sec. 3.1). The harnessing agent recursively revises and updates the primitive implementation, capability state and control logic in recursive self-improvement (RSI) (Sec. 3.2). After the harnessing procedure, we freeze all capability harnesses and save them to the capability harness library. Each capability harness in the library is associated with a brief text description showing the capability summary, input and output information. During test-time task execution, the capability harness library is selectively loaded and reused to complete the task in the test set (Sec. 3.3). The whole capability auto-harnessing pipeline is shown in Fig. 2.

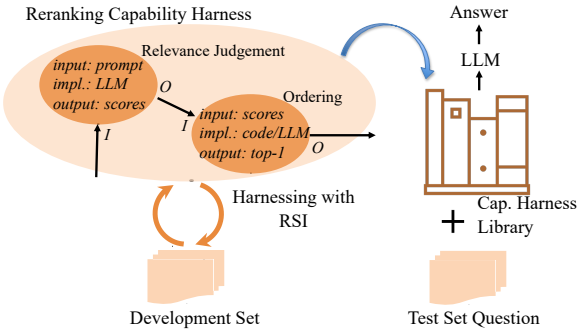

Figure 3: Capability Illustration: we use the *Reranking Capability* as an example. It is divided into *Relevance Judgement* and *Ordering* capability primitives with sequential control logic. The auto-harnessing on development set with recursive self-improvement (RSI) implements the two capability primitives, and finally returns the Reranking Capability Harness which is saved in the capability library.

3.1 Capability in *LongCapHarness*

A capability $\mathcal{C}$ in *LongCapHarness* is defined as an essential *abstract transformation* from the an input $\mathcal{I}$ to an output $\mathcal{O}$,

$$\mathcal{O} = \mathcal{C}(\mathcal{I}), \quad \mathcal{I} \neq \mathcal{O} \tag{1}$$

The capability in Eqn. (1) abstracts the transformation and is independent of a particular Q&A pair. Two Q&A pairs can share the same capability if they require the same transformation. For each capability, *LongCapHarness* constructs its corresponding capability harness: the peripheral concrete, executable workflow that specifies how to perform that transformation. We define the harness as follows,

$$\mathcal{A} \leftarrow \mathcal{H}_{\mathcal{C}}(\mathcal{Q}) \tag{2}$$

where $\mathcal{H}_{\mathcal{C}}$ implements the harness for capability $\mathcal{C}$. It usually includes the code, prompt templates, and execution rules needed to produce the answer. To construct a capability harness, the harnessing agent decomposes the capability $\mathcal{C}$ into multiple capability primitives. Each primitive $\mathcal{C}_{\mathcal{P}_i}$ abstracts a sub-transform and specifies its input I_i and output O_i:

$$\mathcal{C} := \{\mathcal{C}_{\mathcal{P}_1}, \cdots, \mathcal{C}_{\mathcal{P}_m}\}; \quad \mathcal{O}_i \leftarrow \mathcal{C}_{\mathcal{P}_i}(I_i) \tag{3}$$

where $i \in \mathbb{Z}^+$ and $i \leqslant m$. The input to a capability primitive may include the original question, selected task records, or outputs from earlier primitives; its output may be an intermediate result or

part of the final answer. The capability harnessing in Eqn. (2) then becomes capability primitive harnessing and the control logic $\mathcal{F}$ harnessing,

$$\mathcal{A} \leftarrow \mathcal{F}(\{\mathcal{H}_{\mathcal{C}_{\mathcal{P}_i}}(I_i)\}_{i=1}^{m}) \tag{4}$$

We use the *Reranking Capability*, which has been extracted by *LongCapHarness*, to illustrate our capability design in Fig. 3. Based on the development set, the auto-harnessing process identifies the *Reranking Capability* and decomposes it into *Relevance Judgement* primitive and *Ordering* with sequential control logic. The auto-harnessing implements the two capability primitives and recursively improves their harness through recursive self-improvement. The resulting *Reranking Capability Harness* is saved into capability library for potential use on the held-out test set.

3.2 Harnessing Process in *LongCapHarness*

The harnessing agent takes each Q&A pair in the development set as input. By studying the task description from the input question, the harness agent identifies the task's input and output and decides to propose a new capability (see Eqn. (1)) or leverage an existing capability to address the question. Once the required capability is decided, the harnessing agent tries to decompose it into capability primitives (see Eqn. (3)) and identify corresponding the control logic governing those primitives. The harnessing agent then implements each capability primitive so that each primitive becomes an executable operation. Executing those capability primitives with the governing control logic provides us with capability harness execution generated answer. We compare it with the reference answer to continue to revise and improve the capability harness.

In *LongCapHarness*, we adopt recursive self-improvement (RSI (Xia et al., 2026; Chen et al., 2026; Lee et al., 2026; Lin et al., 2026; Lou et al., 2026; Nie et al., 2026)) to continually improve the capability harness. The RSI loop iteratively revises the primitive implementations, their control logic, and the capability state to improve the capability harness.

In *Primitive Implementation*, the harnessing agent implements a capability primitive via either code or an LLM call, depending on the exact primitive as well as its input and output requirement. Code implementation is usually adopted where the primitive execution rules are explicit and without ambiguity, such as assignment, occurrences counting and sorting operations; LLM implementation is suitable when the capability primitive's input and output can be represented by text. The harnessing agent then just requires to specify the text to supply, a prompt template and the response format.

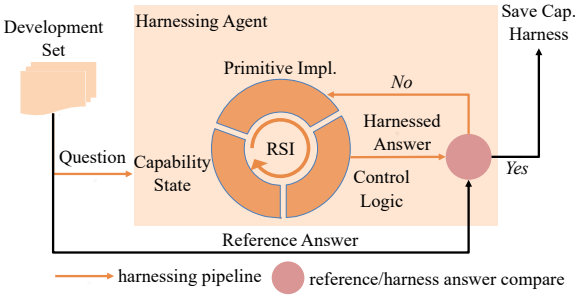

Figure 4: Harnessing Illustration: given the input question, the harnessing agent recursively revise and update the primitive implementation, control logic and capability state until the answer by executing the capability harness is close enough to the reference answer.

In *Control Logic*, the harnessing agent writes the corresponding control logic to combine the capability primitive implementations into a capability harness. The control logic orchestrates the capability primitive operations execution, manage the input/output in-between and further decides the execution stop condition. The capability primitive operations execution governed by control logic generates the capability harness guided answer for the given input question. Comparing it with the reference answer provides important signal to improve the capability harness.

In *Capability State*, the reusable protocol within a capability is treated as the capability state. It defines how each new input question is converted into a format that is directly usable by capability primitives, what intermediate primitive output needs to be retained to derive the final answer. It is worth noting that capability state is agnostic to the actual input from the development set. The per-input runtime state such as the actual question and intermediate results for one input is created fresh during each run and thus does not count as capability state.

In summary, primitive implementations compute results, while control logic schedules and combines those results. The capability state provides the shared workspace that allows these components to exchange outputs without losing the source identity. As shown in Fig. 4, during the capability harnessing with RSI, the harnessing agent recursively revises and updates primitive implementation, control logic and capability state until the difference

between the answer generated by running the capability and the reference answer is small enough (the stop condition is decided by the harnessing agent). The resulting capability harness, including primitive implementation, control logic and capability state, is frozen and saved to the capability library. Each capability harness is associated with a text description summarizing the capability, input and output information, this text description is used during the test-time task execution to select the appropriate capability harness to execute the task.

3.3 Test-Time Task Execution

With the capability harness library obtained in Sec. 3.2, we execute the task in the held-out test set. Given the question from one Q&A pair in the test set, the task execution LLM (we adopt Llama3-8B (Meta, 2024) and Qwen2.5-3B (Qwen et al., 2024)) first selects the capability harness entry by comparing the capability harness text description with the question. After selection, the runtime loads the selected harness's code, prompt templates, and execution rules, and passes the question together with its source text and task information as input to the workflow. The harness is executed as a program, where its control logic initializes fresh per-input state and schedules the capability primitives. For each LLM-based primitive, the harness constructs a bounded prompt from that primitive's template, the question or relevant question fields, and the passages or intermediate results needed for the operation. Code-based primitives operate directly on the stored records and intermediate results. The harness retains these results outside the LLM context and supplies them to subsequent primitives until the workflow produces the final answer.

4 Experiments

We aim to answer four main questions: *Q1:* How well does *LongCapHarness* perform, and what explains its strengths and limitations? *Q2:* Does *LongCapHarness* generalize across different LLM execution models? *Q3:* How does capability primitive RSI affect task performance? *Q4:* How cost-effective is *LongCapHarness*?

4.1 Experimental Setup

Benchmarks. We use HELMET (Yen et al., 2025), RULER (Hsieh et al., 2024) and LongBench-v2 (Bai et al., 2025) to evaluate *LongCapHarness* under different context lengths.

We construct the development set based on the principle of exposing harness development to each evaluated task type while testing reuse on unseen inputs. The resulting split contains 2,242 development examples and 59,826 held-out test examples, with no overlap between the two sets. Appendix C details the development set and test set for each benchmarks.

Models and baselines. We select Llama3-8B (Meta, 2024) and Qwen2.5-3B (Qwen et al., 2024) as execution models for *LongCapHarness* and RRSI (Xia et al., 2026). We compare *LongCapHarness* and RRSI results against the base models and their respective 128K context-extension variants from NExtLong (Gao et al., 2025b) and PolicyLong (Jia et al., 2026b). We also include EntropyLong, using the Llama results from EntropyLong paper (Jia et al., 2025) and the Qwen results from the PolicyLong paper (Jia et al., 2026b), averaged across 8K–128K. During development, both *LongCapHarness* and RRSI use GPT-5.6-Sol as the developer model and Llama3-8B for candidate execution. In RRSI, GPT-5.6-Sol serves as the proposer, selector, critic, and pruner.

Harnessed primitives. LongCapHarness for HELMET and RULER uses keyword matching, lookup, relation following, state tracking, counting, ordering, relevance judgement, rule inference, classification, evidence selection, and evidence integration. Appendix A details the capability decompositions, task mappings, and each primitive's specification, inputs, and outputs.

Metrics and aggregation. We report task scores on a 0–100 scale (the accuracy rate ↑). Metrics include substring exact match for RAG, exact match for ICL and InfiniteBench Choice, NDCG@10 for reranking, and the designated recall or QA metric for RULER. InfiniteBench QA and NarrativeQA use ROUGE-L F1. Category scores average each task across input lengths, then average the tasks within the category.

4.2 Main Results

4.2.1 Comparison of the Three Approaches

Table 1 compares *LongCapHarness* with base models, model-training methods, and RRSI. *LongCapHarness* achieves the highest aggregate score among the evaluated methods for both execution models. With Llama3-8B, *LongCapHarness* outperforms the base model, NExtLong, and RRSI by

Table 1: **Main results on HELMET and RULER.** We compare LongCapHarness, RRSI, and model-training methods with Llama3-8B and Qwen2.5-3B. Bold indicates the highest score among all methods. Dashes indicate that no harness was used or that results were unavailable.

Model	Harness	Context	Avg	H-Avg	Recall	RAG	ICL	Rank	LongQA	RULER
Zero-shot										
Llama3-8B	–	8K	15.14	12.48	18.00	12.68	13.60	7.74	10.38	17.80
Qwen2.5-3B	–	32K	61.91	49.01	–	–	–	–	–	74.81
Data Synthesis + LLM Post-training										
Llama3-8B-NExtLong-128K	–	128K	70.15	58.80	82.56	60.91	81.76	31.47	**37.30**	81.50
Llama3-8B-EntropyLong-128K	–	128K	–	–	–	–	–	–	–	87.37
Qwen2.5-3B-NExtLong-128K	–	128K	62.56	50.26	–	–	–	–	–	74.87
Qwen2.5-3B-EntropyLong-128K	–	128K	62.02	49.11	–	–	–	–	–	74.93
Qwen2.5-3B-PolicyLong-128K	–	128K	64.10	51.53	–	–	–	–	–	76.67
RSI										
Llama3-8B	RRSI	8K	39.67	46.02	28.19	**64.63**	83.45	22.18	31.65	33.32
Qwen2.5-3B		32K	38.46	44.83	39.11	56.88	79.47	21.15	27.55	32.09
Ours										
Llama3-8B	LongCapHarness	8K	**79.32**	**64.01**	**100.00**	59.70	**83.85**	39.67	36.84	**94.62**
Qwen2.5-3B		32K	78.73	63.18	**100.00**	56.33	76.84	**56.28**	26.45	94.28

64.18, 9.17, and 39.65 points. With Qwen2.5-3B, it outperforms the base model, PolicyLong, and RRSI by 16.82, 14.63, and 40.27 points. These results demonstrate the effectiveness of *LongCapHarness* on the evaluated long-context benchmarks.

RRSI, however, falls below NExtLong by 30.48 points with Llama and below PolicyLong by 25.64 points with Qwen. Inspection of RRSI's intermediate development code shows repeated attempts to address context overflow through token limits and character-based chunking. Within the available development rounds, these revisions approach task solving primarily through the physical organization of the input and organize operators around it. In contrast, LongCapHarness starts from the logical organization of task-solving operations. **Q1 Strength:** The capability- decomposition requirements make these operations and their dependencies explicit, guiding RSI to optimize over the task's computational structure.

We next examine the task categories to understand where this approach yields the gains and where its benefits remain limited.

The largest gain appears in Recall, where LongCapHarness achieves 100.00 with both execution models, outperforming RRSI by 71.81 points with Llama and 60.89 points with Qwen, and NExtLong by 17.44 points. Figure 5 shows that this performance persists across the evaluated 8K–128K input lengths. The advantage extends to RULER, where LongCapHarness achieves 94.62 with Llama and 94.28 with Qwen, outperforming RRSI by 61.30 and 62.19 points and the corresponding context-extension models by 13.12 and 17.61 points. **Q1**

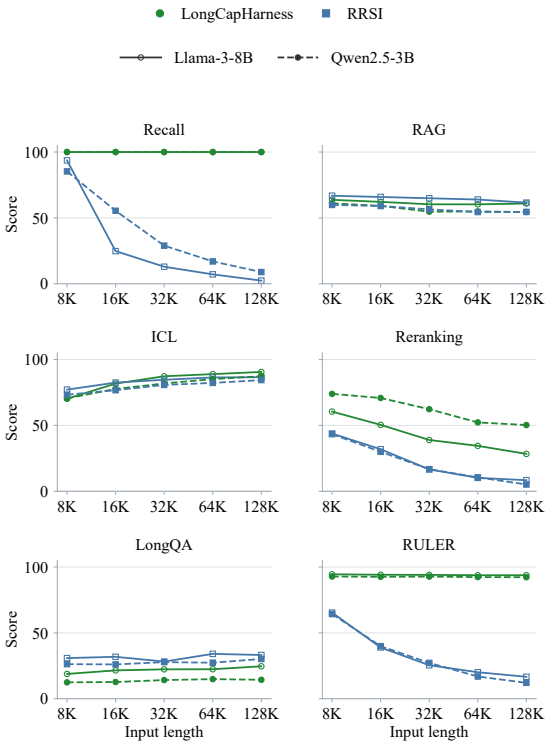

Figure 5: Helmet and Ruler category scores across input lengths with Llama3-8B and Qwen2.5-3B.

Strength: These gains stem from the fact that the underlying primitives have explicit rules that code can execute across longer inputs, without requiring the model to maintain the complete task state in its context. For example, Recall uses keyword matching and lookup, while most RULER tasks use relation following, state tracking, and counting.

The improvements are not limited to tasks that code can solve directly. On reranking, LongCapHarness outperforms RRSI by 17.49 points with Llama and 35.13 points with Qwen; Llama

Table 2: **LongBench-v2 score with Qwen2.5-3B.** Medium and Long denote input lengths of 32K–128K and 128K–2M tokens. Published baseline scores are post-training results from (Jia et al., 2026b).

Method	Medium	Long
Base	21.50	20.60
NExtLong	23.70	26.90
EntropyLong	24.20	28.70
PolicyLong	27.10	**31.50**
LongCapHarness	**34.47**	27.85

also exceeds NExtLong by 8.20 points. **Q1 Strength:** The harness combines model-based relevance judgement with code-controlled ordering, leaving semantic judgements as a limiting factor. Section 4.2.3 examines the effects of different primitive implementations.

The performance gap is smaller and less consistent on ICL, RAG, and LongQA. On ICL, LongCapHarness exceeds RRSI by 0.40 points with Llama and falls below it by 2.63 points with Qwen. On RAG, it trails RRSI by 4.93 and 0.55 points, respectively. On LongQA, Llama improves over RRSI by 5.19 points but remains 0.46 points below NExtLong, while Qwen falls below RRSI by 1.10 points. These mixed gains are consistent with continued reliance on model capabilities: inferring mappings from demonstrations in ICL, and identifying and integrating evidence in RAG and LongQA. **Q1 Limitation:** Harnesses organize these operations but do not replace their semantic judgements.

LongBench-v2. We report our Qwen2.5-3B results on the Medium and Long subsets of LongBench-v2 (Bai et al., 2025) in Table 2, with input lengths of 32K–128K and 128K–2M tokens. LongCapHarness scores 34.47 on Medium and 27.85 on Long, exceeding PolicyLong by 7.37 points on Medium but trailing it by 3.65 points on Long. These results are consistent with the limitation discussed above.

4.2.2 Cross-Model Transfer

Replacing Llama3-8B with Qwen2.5-3B decreases LongCapHarness's Avg by only 0.59 points, from 79.32 to 78.73. The developed harnesses are reused with adaptations to the model, tokenizer, evidence budget, and serving configuration.

Code-based primitives follow the same execution rules across models, as illustrated by Recall remaining at 100.00. Semantic primitives rely on the new model's judgements, leading to task-level differences: reranking improves by 16.61 points with Qwen, while LongQA decreases by 10.39 points. **Q2 Answer:** Cross-model generalization is strong for the code-based components of capability harnesses, while semantic components depend on the execution model's capabilities.

Table 3: **RSI trajectory of HELMET reranking capability primitives.** Bold indicates the highest displayed score in each column.

Version	8K	16K	32K	64K	128K	Mean
Initial code-based implementation (both models)						
V0: BM25 + sort	53.26	45.44	39.52	34.66	28.85	40.35
Llama3-8B						
V1: LLM scoring + sort	65.51	51.88	36.50	22.95	17.89	38.95
V2: Comparison + merge	57.79	47.48	36.70	30.85	25.51	39.67
Qwen2.5-3B						
V1: LLM scoring + sort	68.14	55.13	37.06	25.12	17.08	40.50
V2: Comparison + merge	**71.76**	**64.48**	**55.50**	**45.85**	**43.83**	**56.28**

4.2.3 Capability Primitive Evolution

We examine the RSI trajectory of capability primitives by HELMET Reranking. The target capability remains fixed, while its relevance-judgement and ordering primitives evolve through three implementations: BM25 scoring with numeric sorting (V0), independent LLM scoring with numeric sorting (V1), and relative LLM comparison with group ordering and merging (V2). Table 3 reports their performance across execution models and input lengths.

The four LLM-based runs are evaluated on the same 500 records, with 100 at each input length from 8K to 128K. Each implementation is fixed before evaluation and processes the complete candidate set within the execution model's per-call context budget.

The first evolution, from V0 to V1, changes the relevance-judgement primitive from lexical matching to semantic scoring. The LLM assigns each query–passage pair a relevance grade, while the ordering primitive continues to sort the resulting scores in code. This change improves performance at 8K–16K for both models, while V0 retains higher scores at 32K–128K. Across all lengths, V1 achieves 38.95 with Llama and 40.50 with Qwen, compared with V0's 40.35. This step introduces semantic interpretation into the relevance primitive,

with benefits concentrated at shorter input lengths.

The second evolution, from V1 to V2, changes both the relevance output and its downstream composition. The LLM expresses relative preferences among passages, and the ordering primitive uses these preferences to order small groups and merge them into a complete ranking. With Qwen, this evolution improves the mean by 15.78 points to 56.28 and achieves the highest score at every input length. At 128K, the gain over V1 reaches 26.75 points. With Llama, the mean improves by 0.72 points to 39.67, with gains at 32K–128K; V1 retains higher scores at 8K–16K.

Q3 Answer: These results show that capability primitive RSI improves task performance by jointly refining local judgements and their composition, with gains depending on the execution model and input length.

4.2.4 Cost Effectiveness

We report development and inference usage for *LongCapHarness* and RRSI, alongside model-training rental estimates. Development costs are estimated token charges for GPT-5.6 Sol, and training costs are estimated H100 rental charges. Llama and Qwen inference usage is reported as input and output tokens, and token charges estimated at reference API rates. Development uses GPT-5.6 Sol API rates, inference uses reference API rates, and training uses H100 rental rates. These are estimated charges; pricing sources and assumptions are given in Appendix B.

Development costs. Table 4 shows that LongCapHarness uses 814.11M input and 3.35M output tokens during development, compared with 52.42M and 3.87M for RRSI, yielding estimated API charges of $541.47 and $331.56.

NExtLong's reported training configuration costs an estimated $3,696.00. PolicyLong's FLOP-scaled estimate is $1,858.82, assuming equal effective throughput per GPU. Overall, LongCapHarness has higher development API charges than RRSI but lower estimated costs than either model-training reference.

Inference costs. We compare LongCapHarness and RRSI on the same benchmark test set. Table 4 shows that LongCapHarness uses more input tokens but fewer output tokens with both models. At the reference API rates, its estimated charges are slightly lower with Llama ($8.23 vs. $8.30) but higher with Qwen ($53.74 vs. $29.39).

Table 4: **Development, training, and inference costs.** Input and output are in millions of tokens; costs are estimated USD. Training rows show H100 GPU-hours and rental costs; PolicyLong uses the FLOP-scaling estimate. Bold indicates lower token usage or cost within each group; pink highlights LongCapHarness's better values.

Method	H100 GPU-hours		Cost ($)
Model Training			
NExtLong (Gao et al., 2025b)	960		3,696.00
PolicyLong (Jia et al., 2026b)	482.81		**1,858.82**
Method	**Input (M)**	**Output (M)**	**Cost ($)**
Development (GPT-5.6 Sol)			
RRSI (Xia et al., 2026)	**52.42**	3.87	**331.56**
LongCapHarness	814.11	**3.35**	541.47
Inference (Llama-3-8B)			
RRSI (Xia et al., 2026)	**236.05**	89.55	8.30
LongCapHarness	408.00	**1.87**	**8.23**
Inference (Qwen2.5-3B)			
RRSI (Xia et al., 2026)	**285.61**	8.33	**29.39**
LongCapHarness	535.08	**2.34**	53.74

Q4 Answer: In conclusion, LongCapHarness invests more development tokens in finding, testing, and freezing reusable code implementations in the capability harness. During inference, these implementations execute operations that would otherwise require model reasoning, reducing generated output tokens.

5 Conclusion

LongCapHarness decomposes long-context tasks into capability primitives and uses RSI to build reusable harnesses without data synthesis or model post-training. Transferred from Llama3-8B to Qwen2.5-3B, it improves average HELMET and RULER scores over model-training baselines; its estimated development cost is below the estimated training costs for NExtLong and PolicyLong. These results support harnessing latent LLM capabilities beyond explicit training.

6 Limitation

LongCapHarness still relies on a powerful harnessing agent (GPT-5.6-Sol) to construct its capability harnesses. We observe a sharp performance drop when weaker models such as Llama3-8B and Qwen2.5-3B serve as their own harnessing agents. Enabling weaker models to develop effective harnesses independently remains an open challenge.

7 Ethical Considerations

Our method constructs and refines capability harnesses for long-context reasoning without additional LLM training. All benchmark datasets used in this study are publicly available and contain no personal information. No human subjects are involved, and all LLMs used are publicly available models or accessible through public APIs. We do not identify additional ethical concerns specific to the proposed method within this experimental setting.

References

Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2025. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3639–3664.

Jianghao Chen, Junhong Wu, Yangyifan Xu, and Jiajun Zhang. 2025. Ladm: Long-context training data selection with attention-based dependency measurement for llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3076–3090.

T. Chen, S. Lu, K. Zhao, W. Meng, H. Teng, T. Li, C. Li, X. Liu, J. Liang, Z. Zhang, et al. 2026. HarnessX: A composable, adaptive, and evolvable agent harness foundry. *Preprint*, arXiv:2606.14249.

Haoran Deng, Yingyu Lin, Zhenghao Lin, Xiao Liu, Yizhou Sun, Yian Ma, and Yeyun Gong. 2026. Beyond length: Quantifying long-range information for long-context LLM pretraining data. In *The Fourteenth International Conference on Learning Representations*.

Chaochen Gao, Qi Fu, Songlin Hu, et al. 2025a. Quest: Query-centric data synthesis approach for long-context scaling of large language model. In *International Conference on Learning Representations*, volume 2025, pages 91812–91832.

Chaochen Gao, Xing Wu, Zijia Lin, Debing Zhang, and Songlin Hu. 2025b. NExtLong: Toward Effective Long-Context Training Without Long Documents. In *International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 18566–18584. PMLR.

Tianyu Gao, Alexander Wettig, Howard Yen, and Danqi Chen. 2025c. How to train long-context language models (effectively). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7376–7399.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's The Real Context Size Of Your Long-Context Language Models? In *Annual Conference on Language Modeling (CoLM)*.

Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated design of agentic systems. *Preprint*, arXiv:2408.08435.

Junlong Jia, Ziyang Chen, Xing Wu, Chaochen Gao, Zijia Lin, Debing Zhang, Songlin Hu, and Binghui Guo. 2025. Entropylong: Effective long-context training via predictive uncertainty. *Preprint*, arXiv:2510.02330.

Junlong Jia, Xing Wu, Chaochen Gao, Ziyang Chen, Zijia Lin, Zhongzhi Li, Weinong Wang, Haotian Xu, Donghui Jin, Debing Zhang, et al. 2026a. Litelong: Resource-efficient long-context data synthesis for llms. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.

Junlong Jia, Jiang Zhou, Ziyang Chen, Xing Wu, Chaochen Gao, Tinghao Yu, Feng Zhang, and Songlin Hu. 2026b. PolicyLong: Towards On-Policy Context Extension. In *Annual Conference on Language Modeling (CoLM)*.

Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. 2026. Meta-Harness: End-to-end optimization of model harnesses. In *The Third Conference on Language Modeling*.

J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, et al. 2026. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *Preprint*, arXiv:2604.25850.

Shicheng Liu, Jialiang Xu, Wesley Tjangnaka, Sina Semnani, Chen Yu, and Monica Lam. 2024. SUQL: Conversational search over structured and unstructured data with large language models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 4535–4555.

Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. 2026. AutoHarness: Improving LLM agents by automatically synthesizing a code harness. *Preprint*, arXiv:2603.03329.

Meta. 2024. Meta Llama 3 model card. Original April 2024 release.

Nebius. 2026. Compute Pricing in Nebius AI Cloud. `https://docs.nebius.com/compute/resources/pricing/`. NVIDIA H100 NVLink on-demand pricing in USD per GPU-hour, eu-north1. Accessed September 17, 2026.

J. Nie, Y. Zhang, J. Song, Q. Cai, D. Yu, Y. Guo, X. Tian, and B. Han. 2026. TTHE: Test-time harness evolution. *Preprint*, arXiv:2607.08124.

OpenAI. GPT-5.6 Sol. https://developers.openai.com/api/docs/models/gpt-5.6-sol. Accessed October 13, 2026.

OpenAI. 2026. API Pricing. https://developers.openai.com/api/docs/pricing. GPT-5.6 Sol, Standard processing rates. Accessed September 17, 2026.

OpenRouter. 2026a. Meta: Llama 3.1 8B Instruct. https://openrouter.ai/meta-llama/llama-3.1-8b-instruct. DeepInfra input/output rates: USD 0.02/0.04 per million tokens. Price quotation provided October 4, 2026.

OpenRouter. 2026b. Qwen2.5 Coder 3B. Input/output rates: USD 0.10/0.10 per million tokens. Price quotation supplied by the manuscript author on October 4, 2026.

Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS. *Proceedings of the VLDB Endowment*, 18(11):4171–4184.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, and 24 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. 2026. Rrsi: Regularized recursive self-improvement of agent harnesses. *Preprint*, arXiv:2609.24972.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2025. HELMET: How To Evaluate Long-Context Language Models Effectively And Thoroughly. In *International Conference on Learning Representations (ICLR)*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2024. AFlow: Automating agentic workflow generation. *Preprint*, arXiv:2410.10762.

A Harnessed Capabilities and Primitives

The following list summarizes the capability decompositions described for our HELMET and RULER harnesses. Capabilities are grouped by the transformation required to solve a task. Primitive names describe transformations, not particular code or prompt implementations. Multiple task families can share a decomposition.

- **Matching and retrieval.** *Keyword matching* and *lookup* of matching records or values support HELMET Recall and RULER needle retrieval.
- **Relation and state tracking.** *Relation following* across records and *state tracking* through updates support RULER variable tracking.
- **Frequency-based selection.** *Counting* occurrences and *ordering* by frequency support RULER common/frequent word extraction.
- **Passage reranking.** *Relevance judgement* and *ordering* candidates support HELMET Reranking.
- **Example-based classification.** *Rule inference* from examples and *classification* into allowed labels support HELMET ICL.
- **Evidence-based answering.** *Evidence selection* of relevant passages and *evidence integration* into an answer support HELMET RAG and LongQA, and RULER QA.

A.1 Primitive Specifications and Interfaces

For each primitive, *Spec* states the intended transformation, while *Input* and *Output* define its interface. These are abstract requirements, not implementation-specific data formats or guarantees of model accuracy.

- **Keyword matching.** *Spec:* Locate occurrences satisfying the supplied lexical matching rules. *Input:* Keywords, text records, and matching rules. *Output:* Matching spans and their record identities.
- **Lookup.** *Spec:* Retrieve records or values satisfying a search condition. *Input:* Search condition and records. *Output:* Matching records or values.
- **Relation following.** *Spec:* Follow the specified relations from a starting entity across records to recover the requested entity or value. *Input:* Starting entity, relations to follow, and records. *Output:* Related entity or value.
- **State tracking.** *Spec:* Apply events according to the supplied update rules and event order to determine the state at a requested time. *Input:* Initial state, events, update rules, and

requested time. *Output:* State at the requested time.

- **Counting.** *Spec:* Aggregate occurrences according to the supplied counting rules. *Input:* Records and counting rules. *Output:* A count or per-item counts.
- **Ordering.** *Spec:* Arrange candidates according to the supplied scores, preferences, or ordering rules while preserving candidate identities. *Input:* Candidates and ordering rules, including any required scores or comparison outputs. *Output:* Ordered candidate IDs.
- **Relevance judgement.** *Spec:* Assess how well a passage addresses the query, or compare the relevance of candidate passages. *Input:* Query and passage(s). *Output:* Relevance score or passage preference.
- **Rule inference.** *Spec:* Infer a mapping compatible with the supplied input–output examples and any candidate-rule constraints. *Input:* Input–output examples and candidate rules, when supplied. *Output:* Inferred mapping or compatible rule(s).
- **Classification.** *Spec:* Assign an input to an allowed label using the supplied examples or inferred mapping. *Input:* Input to classify, allowed labels, and examples or mapping. *Output:* Predicted label.
- **Evidence selection.** *Spec:* Identify passages or spans relevant to answering the question while retaining their source identities. *Input:* Question, candidate passages, and source IDs. *Output:* Selected evidence with source IDs.
- **Evidence integration.** *Spec:* Combine information from the supplied evidence to answer the question, preserving the relationships needed for the answer. *Input:* Question, selected passages or evidence, and source IDs. *Output:* Answer.

B Cost Estimation Details

API-price estimates. For development, we apply GPT-5.6 Sol Standard short-context rates of \$4/\$0.40/\$20 per million ordinary-input/cache-read/output tokens for requests with at most 272K input tokens (OpenAI, 2026). For inference, we price Llama3-8B at \$0.02/\$0.04 per million input/output tokens using DeepInfra's Llama 3.1 8B Instruct quotation on OpenRouter (OpenRouter, 2026a), and Qwen2.5-3B at \$0.10/\$0.10 using the Qwen2.5 Coder 3B quotation on OpenRouter (OpenRouter, 2026b).

Training rental estimates. At an H100 rental rate of \$3.85 per GPU-hour (Nebius, 2026), NExtLong's reported 64 GPUs for 15 hours (Gao et al., 2025b) cost an estimated \$3,696.00. Scaling by approximate training FLOPs $F \approx 6T(P + Lsd)$, where T is the token budget, P the parameter count, L the layer count, s the sequence length, and d the hidden width, and assuming equal effective throughput per GPU gives a PolicyLong estimate of \$1,858.82.

Table 5: Development and test/evaluation records by benchmark.

Benchmark	Development	Test/evaluation	Total
HELMET	2,080	27,035	29,115
RULER	112	32,388	32,500
LongBench-v2	50	403	453
Total	2,242	59,826	62,068

Table 6: Full HELMET/RULER split by category, aggregated over five input lengths.

Category	Development	Test	Total
Recall	70	1,930	2,000
RAG	945	9,555	10,500
ICL	75	12,425	12,500
Re-rank	210	405	615
LongQA	450	1,050	1,500
Citation	30	970	1,000
Summarization	300	700	1,000
HELMET subtotal	2,080	27,035	29,115
RULER	112	32,388	32,500
Total	2,192	59,423	61,615

C Development and Evaluation Data

Table 5 breaks down the 2,242 development and 59,826 test/evaluation records reported in Section 4.1 by benchmark. The following tables provide category, subtask, and input-length breakdowns.

C.1 HELMET and RULER

Both partitions cover 34 subtasks at five benchmark input lengths, or 170 task–length cells each. These lengths describe benchmark inputs, not the context budget of an individual LLM call. Tables 7 and 8

Table 7: Full HELMET/RULER split by benchmark input length.

Input length	Development	Test	Total
8K (8,192)	458	11,865	12,323
16K (16,384)	431	11,892	12,323
32K (32,768)	438	11,885	12,323
64K (65,536)	433	11,890	12,323
128K (131,072)	432	11,891	12,323
Total	2,192	59,423	61,615

Table 8: All 34 HELMET/RULER subtasks in the full split.

Category	Subtask	Dev.	Test	Total
Recall	json_kv	54	446	500
Recall	niah_multikey_2	5	495	500
Recall	niah_multikey_3	5	495	500
Recall	niah_multivalue	6	494	500
RAG	rag_hotpotqa	45	1,455	1,500
RAG	rag_natural_questions	90	2,910	3,000
RAG	rag_popqa	720	2,280	3,000
RAG	rag_triviaqa	90	2,910	3,000
ICL	icl_banking77	15	2,485	2,500
ICL	icl_clinic150	15	2,485	2,500
ICL	icl_nlu	15	2,485	2,500
ICL	icl_trec_coarse	15	2,485	2,500
ICL	icl_trec_fine	15	2,485	2,500
Re-rank	rerank_msmarco	210	405	615
LongQA	infbench_choice	155	345	500
LongQA	infbench_qa	145	355	500
LongQA	narrativeqa	150	350	500
Citation	alce_asqa	15	485	500
Citation	alce_qampari	15	485	500
Summarization	infbench_sum_eng	150	350	500
Summarization	multi_lexsum	150	350	500
RULER	ruler_cwe	5	2,495	2,500
RULER	ruler_fwe	5	2,495	2,500
RULER	ruler_niah_multikey_1	8	2,492	2,500
RULER	ruler_niah_multikey_2	5	2,495	2,500
RULER	ruler_niah_multikey_3	5	2,495	2,500
RULER	ruler_niah_multiquery	7	2,493	2,500
RULER	ruler_niah_multivalue	7	2,493	2,500
RULER	ruler_niah_single_1	13	2,487	2,500
RULER	ruler_niah_single_2	14	2,486	2,500
RULER	ruler_niah_single_3	7	2,493	2,500
RULER	ruler_qa_1	15	2,485	2,500
RULER	ruler_qa_2	15	2,485	2,500
RULER	ruler_vt	6	2,494	2,500
Total	34 subtasks	2,192	59,423	61,615

report the length and subtask breakdowns; the latter sums over all five lengths.

C.2 LongBench-v2

Table 9 breaks down the 50 development and 403 held-out evaluation records across five non-code domains, with ten development records per domain.

Table 9: LongBench-v2 split by domain. Test records exclude the development subset.

Domain	Development	Test	Total
Single-Document QA	10	165	175
Multi-Document QA	10	115	125
Long In-context Learning	10	71	81
Long-dialogue History Understanding	10	29	39
Long Structured Data Understanding	10	23	33
Total	50	403	453